

LLM 应用开发 + Agent 智能体开发 完整分阶学习路线

☑ 重要内容	☐
🕒 类型	笔记
🏷 标签	Empty
📌 状态	Empty
📁 项目	Empty
📁 领域	Empty
🌐 网址	Empty
☑ 归档	☐
🕒 创建时间	2026/07/01 12:27

LLM 应用开发 + Agent 智能体开发 完整分阶学习路线

前置说明

1. 关系：Agent 开发 = LLM 应用开发全套基础 + Agent 专属进阶内容，两条路线递进学习；
2. 技术栈区分：
 - LLM 应用：Python 为主，少量前端；
 - Agent（你关注的 Pi / Hermes）：Python + TypeScript 双栈，偏底层运行时、记忆、调度；
3. 目标分层：
 - LLM 应用：快速做问答、知识库、固定流程 AI 工具；
 - Agent 开发：自主规划、循环执行、长期记忆、本地自治数字助手。

第一部分：通用基础（两条路线都必须学，0 基础起步）

阶段 1：编程 & 后端工程打底（1~2 周）

Python 核心

1. 基础语法、类、异常、文件 IO、异步 `async/await`
2. Pydantic 数据校验（LLM 结构化输出必备）
3. 第三方库：requests、python-dotenv、json 处理

TypeScript / Node.js（想上手 Pi 框架必学）

1. TS 类型、异步、模块化、npm 包开发
2. 事件、RPC 基础、CLI 程序开发（Pi 是 TS Agent 运行时）

Web 服务基础

1. FastAPI 快速接口、流式 SSE、WebSocket 对话推送
2. 并发、限流、CORS、日志

工程部署通用

1. Git、Docker 容器打包
2. Linux 基础命令（Agent 会大量调用 Shell 工具）

阶段 2：LLM 底层通用知识（2~3 周，LLM/Agent 共用）

1. 大模型调用基础

- 主流 API：OpenAI、Claude、通义千问、Ollama 本地开源模型
- 流式输出、上下文窗口、token 计算、成本控制

2. Prompt 工程（核心基础）

- 基础：角色提示、Few-shot、分隔符防注入
- 进阶：CoT 思维链、结构化输出 JSON、格式约束

3. Function Calling / 工具调用基础

- 函数定义、参数描述、模型自动判断调用工具
- 工具返回结果二次喂给 LLM 形成简单闭环

4. RAG 检索增强（所有 LLM 应用标配）

1. 文本处理：分块、清洗、摘要
2. 嵌入模型：text-embedding、bge、m3e
3. 向量库：Chroma、PgVector、FAISS、Milvus
4. 检索优化：关键词 + 向量混合检索、重排 Rerank

5. 简单对话会话管理

内存 / 数据库存储历史、窗口裁剪、多用户隔离

阶段 3：LLM 应用开发专项路线（学完可独立交付 AI 业务系统）

适合：知识库问答、智能客服、文档总结、AI 写作、企业固定流程助手

阶段 3-1：低代码快速落地（1 周）

Dify / FastGPT / Coze 平台使用：

- 搭建知识库、提示词编排、API 发布、插件接入

阶段 3-2：自研 LLM 应用全栈（2~4 周）

1. 基础对话系统：多轮聊天、会话持久化
2. 垂直 RAG 系统：企业文档解析 (PDF/Word/Markdown)、私有知识库
3. 线性固定 Workflow（无自主循环）LangChain 基础链：RetrievalQA、SequentialChain、路由链
4. 多模态应用：图文输入、图片理解、文档 OCR+LLM 总结
5. 前端配套：Vue/HTML 聊天页面、流式渲染对话

阶段 3-3：LLM 应用进阶优化（可选）

1. 领域微调入门：LoRA、QLoRA、数据集构建
2. 评测体系：问答准确率、幻觉检测
3. 私有化部署：本地 Ollama、模型服务封装

LLM 应用阶段产出项目

- 1. 企业内部文档问答机器人
- 2. 智能客服多轮对话系统
- 3. PDF 批量解析总结工具
- 4. 文案生成小程序

第二部分：Agent 智能体开发专属进阶（在 LLM 基础之上额外学习）

适用场景：Pi 框架、Hermes 记忆引擎、编码 Agent、自主数字员工、多步骤自动化任务

阶段 4：Agent 理论与基础范式（1~2 周）

- 1. 核心范式
 - ReAct、Plan & Solve、Self-Reflection 反思机制
 - Agent 循环：感知 → 规划 → 工具执行 → 观察反思 → 终止判断
- 2. 基础框架入门
 - LangGraph（状态机式 Agent 标准）
 - AutoGen（多智能体基础）
- 3. 关键概念
 - 任务状态、断点 Checkpoint、迭代次数限制、终止条件
 - Human-in-the-Loop 人工介入节点

阶段 5：记忆系统专项（Hermes 核心，2~3 周）

- 1. 分层记忆架构
 - 短期上下文记忆
 - 长期向量记忆（跨会话持久存储）
 - 技能库记忆、用户画像记忆
- 2. 记忆能力开发
 - 记忆压缩、自动摘要、相关性检索

- 会话分支、回滚、记忆清洗

3. 实践：Hermes Memory Engine 集成、Mem0 使用

阶段 6：Agent 运行时基座开发（对应 earendil-works/pi，3~4 周）

1. Pi 框架完整学习（TS 栈）

1. Pi 整体架构：Agent Harness 运行时、模块化 Monorepo

2. 核心包拆解

- @earendil-works/pi-ai：多模型统一抽象层
- @earendil-works/pi-agent-core：智能体循环内核
- pi-hermes-memory 插件对接 Hermes 记忆

3. 扩展开发：自定义 Pi 插件、技能包

2. 多端交互开发

- TUI 终端交互界面、WebUI 网页面板
- RPC 跨进程调用，支持其他语言调用 Agent

3. Agent 安全沙箱（重点）

文件读写权限管控、Shell 命令隔离、操作审批、密钥防护

阶段 7：高级智能体系统（2~3 周）

1. 任务规划深度开发自动任务拆解、子任务调度、失败重试、分支判断
2. 多智能体协同编排 Agent + 执行 Worker 分工、Agent 间通信
3. 自主进化闭环（Hermes GEPA）任务执行 → 评估 → 沉淀可复用技能 → 下次自动复用
4. 复杂工具链封装文件操作、Git、数据库、API 批量调用、定时任务

阶段 8：Agent 工程化部署（1 周）

1. 后台常驻 Agent 服务、开机自启
2. 容器打包完整 Pi+Hermes 环境
3. 监控、任务日志、异常自动恢复

Agent 阶段完整产出项目

1. 基于 Pi + Hermes 的本地编码智能体
2. 自动化办公数字员工（批量处理表格、发邮件、爬取数据）
3. 多智能体数据分析系统
4. 运维自动化 Agent（服务器巡检、日志分析）

第三部分：两条路线异同对比总结

一、相同学习内容（全部通用基础）

1. Python/TS、FastAPI、Docker 等工程基础
2. LLM 调用、Prompt、Function Calling
3. RAG 向量检索、对话历史管理
4. 模型私有化、接口部署、并发优化

二、LLM 应用独有内容（Agent 不用深挖）

1. Dify/FastGPT 低代码平台
2. 线性固定 LangChain 链
3. 纯前端 AI 聊天页面
4. 单轮 / 多轮静态问答系统
5. 轻量化领域微调

三、Agent 开发独有内容（LLM 应用不需要）

1. ReAct / 反思 / Agent 循环、LangGraph 状态机
2. 分层长期记忆引擎（Hermes）
3. Agent 运行时基座（Pi 框架）、插件体系
4. 任务动态规划、断点续跑、多智能体协作
5. 工具沙箱、终端 TUI、RPC 跨进程通信
6. 自主技能沉淀、自进化闭环

第四部分：两条路线学习路径推荐

路径 1：只做 LLM 业务应用（周期短，就业面广）

通用基础 → LLM 专项路线 → 就业落地适合：AI 产品、企业知识库、智能客服、ToC 对话工具开发

路径 2：Agent 全栈开发（你当前研究 Pi/Hermes 推荐路线）

通用基础 → LLM 专项路线（必学打底） → Agent 理论与记忆 → Pi 运行时框架 → 高级多智能体系统适合：本地自治智能体、编码助手、自动化数字员工、底层 Agent 框架二次开发

学习周期参考

- 仅 LLM 应用开发：1~2 个月可独立交付项目
- LLM + Agent 完整全栈：3~5 个月，可自研 Agent 基座、对接 Hermes 记忆